Authentication

Secure User Auth from Registration to Sessions

■ LEARNING OUTCOME

By the end of this module you will implement complete user authentication with JWT, bcrypt password hashing, refresh token rotation, OAuth social login, and role-based access control.

■ Skills You Will Gain
• Hash passwords securely with bcrypt
• Generate and verify JWT access and refresh tokens
• Implement register, login, logout, and token refresh flows
• Protect routes with authentication middleware
• Add Google OAuth social login
• Implement role-based access control (RBAC)

■ **Authentication is the most security-critical part of any application. One mistake can expose every user's data. Understand JWT, bcrypt, refresh tokens, and OAuth before building any production auth system.**

SECTION 1

Password Hashing with bcrypt

Why bcrypt -- Never Store Plain Passwords

Never store: plain text, MD5, SHA1, or SHA256 -- all crackable. Use: bcrypt, Argon2, or scrypt -- designed for passwords. bcrypt has a 'cost factor' (work factor) that makes hashing slow on purpose. cost=12 takes ~250ms. Cost=14 takes ~1000ms. This makes brute-force attacks impractical -- 1000 ms per guess = very slow attacker. Salt: bcrypt auto-generates a unique random salt per password. Result: same password hashed twice gives different hash -- rainbow tables useless.

```
npm install bcryptjs jsonwebtoken // services/authService.js const bcrypt = require('bcryptjs'); const jwt = require('jsonwebtoken'); const prisma = require('../config/db'); const AppError = require('../utils/AppError'); const SALT_ROUNDS = 12; const ACCESS_EXPIRES = '15m'; const REFRESH_EXPIRES = '7d'; // REGISTER exports.register = async ({ name, email, password }) => { // Check if email already exists const existing = await prisma.user.findUnique({ where: { email } }); if (existing) throw new AppError('Email already registered', 409); // Hash password -- bcrypt is intentionally slow const hashedPassword = await bcrypt.hash(password, SALT_ROUNDS); const user = await prisma.user.create({ data: { name, email, password: hashedPassword }, select: { id:true, name:true, email:true, role:true }, // never return password! }); const tokens = generateTokens(user); await storeRefreshToken(user.id, tokens.refreshToken); return { user, ...tokens }; }; // LOGIN exports.login = async ({ email, password }) => { const user = await prisma.user.findUnique({ where: { email } }); // Same error for wrong email AND wrong password (prevent user enumeration) if (!user || !(await bcrypt.compare(password, user.password))) { throw new AppError('Invalid email or password', 401); } const { password: _, ...safeUser } = user; // remove password from object const tokens = generateTokens(safeUser); await storeRefreshToken(user.id, tokens.refreshToken); return { user: safeUser, ...tokens }; }; // TOKEN GENERATION const generateTokens = (user) => ({ accessToken: jwt.sign( { id: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: ACCESS_EXPIRES } ), refreshToken: jwt.sign( { id: user.id }, process.env.JWT_REFRESH_SECRET, { expiresIn: REFRESH_EXPIRES } ), }); // REFRESH TOKEN ROTATION exports.refreshTokens = async (refreshToken) => { let payload; try { payload = jwt.verify(refreshToken, process.env.JWT_REFRESH_SECRET); } catch { throw new AppError('Invalid or expired refresh token', 401); } // Verify token is in database (rotation: old token is invalidated) const stored = await prisma.refreshToken.findUnique({ where: { token: refreshToken } }); if (!stored || stored.revoked) throw new AppError('Refresh token revoked', 401); // Revoke old token, issue new pair await prisma.refreshToken.update({ where:{id:stored.id}, data:{revoked:true} }); const user = await prisma.user.findUnique({ where:{id:payload.id} }); const tokens = generateTokens(user); await storeRefreshToken(user.id, tokens.refreshToken); return tokens; }; // LOGOUT exports.logout = async (refreshToken) => { await prisma.refreshToken.updateMany({ where: { token: refreshToken }, data: { revoked: true }, }); };
```

SECTION 2

Auth Routes and Frontend Integration

```
// routes/auth.js router.post('/register', authValidation, validate, wrap(authCtrl.register)); router.post('/login', authValidation, validate, wrap(authCtrl.login)); router.post('/refresh', wrap(authCtrl.refresh)); router.post('/logout', protect, wrap(authCtrl.logout)); router.get ('/me', protect, wrap(authCtrl.getMe)); // controllers/authController.js exports.login = async (req, res) => { const { user, accessToken, refreshToken } = await authService.login(req.body); // Store refresh token in httpOnly cookie (not localStorage -- XSS safe) res.cookie('refreshToken', refreshToken, { httpOnly: true, secure: process.env.NODE_ENV === 'production', sameSite: 'strict', maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days in ms }); res.json({ user, accessToken }); // short-lived access token in response body }; exports.refresh = async (req, res) => { const refreshToken = req.cookies.refreshToken; if (!refreshToken) return res.status(401).json({ error: 'No refresh
```

```
token' }); const tokens = await authService.refreshTokens(refreshToken); res.cookie('refreshToken', tokens.refreshToken, { httpOnly:true, secure:true, sameSite:'strict', maxAge:7*24*60*60*1000 }); res.json({ accessToken: tokens.accessToken }); }; // frontend/src/context/AuthContext.jsx import { createContext, useContext, useState, useCallback } from 'react'; import { api } from '../services/api'; const AuthCtx = createContext(null); export const useAuth = () => useContext(AuthCtx); export function AuthProvider({ children }) { const [user, setUser] = useState(null); const [token, setToken] = useState(null); // access token in memory only! const login = async (credentials) => { const { data } = await api.post('/auth/login', credentials); setToken(data.accessToken); setUser(data.user); }; const logout = useCallback(async () => { await api.post('/auth/logout'); setToken(null); setUser(null); }, []); // Axios interceptor: add token + auto-refresh on 401 // (configured in api.js using token from closure) return <AuthCtx.Provider value={{ user, token, login, logout, isAuth:!!token }}>{children}</AuthCtx.Provider>; }
```

SECTION 3

Google OAuth with Passport.js

```
npm install passport passport-google-oauth20 express-session // config/passport.js const passport = require('passport'); const { Strategy: GoogleStrategy } = require('passport-google-oauth20'); const { findOrCreateGoogleUser } = require('../services/authService'); passport.use(new GoogleStrategy({ clientID: process.env.GOOGLE_CLIENT_ID, clientSecret: process.env.GOOGLE_CLIENT_SECRET, callbackURL: '/api/auth/google/callback', }, async (accessToken, refreshToken, profile, done) => { try { const user = await findOrCreateGoogleUser({ googleId: profile.id, email: profile.emails[0].value, name: profile.displayName, avatar: profile.photos[0].value, }); done(null, user); } catch (err) { done(err, null); } })); // routes/auth.js router.get('/google', passport.authenticate('google', { scope: ['profile','email'] })); router.get('/google/callback', passport.authenticate('google', { session:false, failureRedirect: '/login?error=oauth' }), (req, res) => { const tokens = generateTokens(req.user); // Redirect to frontend with token in query (or via httpOnly cookie)
res.redirect(`${process.env.CLIENT_URL}/auth/callback?token=${tokens.accessToken}`); }); // .env -- get credentials from console.cloud.google.com
GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com GOOGLE_CLIENT_SECRET=your-client-secret
```

■ PRACTICE Q & A

Q1. Why is bcrypt preferred over SHA256 for password hashing?

> ***A:*** *SHA256 is designed to be FAST -- millions of hashes/second. Attackers can brute-force quickly. bcrypt is deliberately SLOW (250ms at cost=12) with a cost factor. Even if the database is stolen, brute-forcing is impractical.*

Q2. What is the difference between an access token and a refresh token?

> ***A:*** *Access token: short-lived (15 minutes), sent in Authorization header for every request. Refresh token: long-lived (7 days), stored in httpOnly cookie, used ONLY to get a new access token. If access token is stolen, it expires quickly.*

Q3. Why store the access token in memory instead of localStorage?

> ***A:*** *localStorage is accessible via JavaScript -- XSS attacks can steal it. Memory (React state) is not accessible via JS from another origin. The refresh token in httpOnly cookie also cannot be read by JS -- immune to XSS.*

Q4. What is refresh token rotation?

> ***A:*** *When a refresh token is used, it is invalidated and a new one is issued. If an attacker steals a refresh token and uses it, the legitimate user's next use will detect the reuse (old token already invalidated) and invalidate all sessions.*

Q5. What is the same-origin policy and CORS?

> ***A:*** *Same-origin: browsers block JS from reading responses from different origins. CORS headers from the server explicitly allow specific origins. Without CORS config, your frontend cannot call your API from a different domain.*

■ Auth Cheat Sheet	
`bcrypt.hash(password, 12)`	Hash password before storing
`bcrypt.compare(input, hash)`	Verify password on login
`jwt.sign(payload, secret, opts)`	Create JWT token

`jwt.verify(token, secret)`	Verify and decode JWT
`{ expiresIn: '15m' }`	Access token: short expiry
`{ expiresIn: '7d' }`	Refresh token: longer expiry
`httpOnly: true`	Cookie not readable by JS
`secure: process.env.NODE_ENV==='production'`	HTTPS only in prod
`sameSite: 'strict'`	CSRF protection for cookies
`same error for wrong email/pw`	Prevent user enumeration
`never return password field`	`select: { password: false }`
`refresh token rotation`	Invalidate on every use